Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

May 6, 2020 · 11 min read · ✦ · Listen

Save

SPACE SCIENCE WITH PYTHON

Space Science with Python — Space maps

Part 5 of the tutorial series shows how to calculate and understand coordinate systems that are used for all upcoming science topics

Photo by Conner Baker on Unsplash

15 | 1

Preface

This is the 5th part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!

Introduction

In our last tutorials we computed the position and velocity vectors of different celestial objects. We determined the apparent angular distance between objects (so called phase-angle) and worked on some small projects using Python and the NASA library SPICE (using the SPICE wrapper *spiceypy*).

In the next couple of weeks we will learn and work together on (scientific) data and larger projects that will cover e.g.:

- Asteroids, Near-Earth Objects and Meteors
- The Rosetta Mission and the descent of Philae on 67P/Churyumov–Gerasimenko
- The Cassini Mission, especially the Cosmic Dust Analyzer, where we will also apply Machine Learning algorithms for data reduction and analysis

Before we start with our scientific deep dives we need to cover 2 fundamental topics:

- Coordinate Systems / Reference Frames
- Orbital Mechanics

It may sound like dry theory, but we will work closely with Python examples. With a solid knowledge fundament and understanding of both topics, in combination with SPICE and other tools that will follow, you will be prepared to achieve great scientific insights!

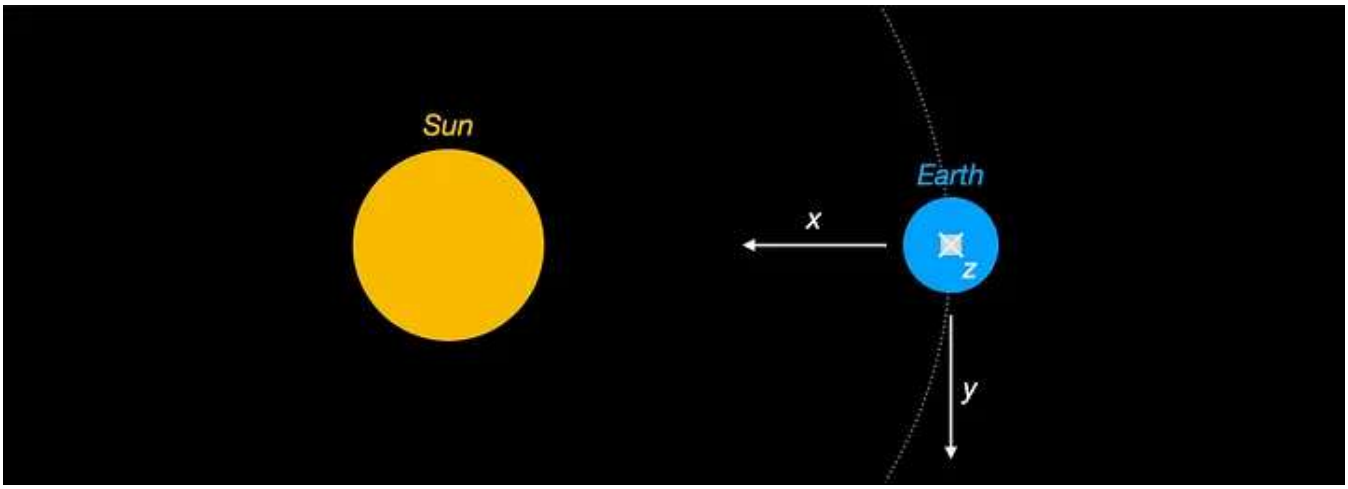
Ecliptic Coordinate System

A 3-dimensional coordinate system has an x, y and z component. All three components are aligned orthogonally, so the angle between all components is 90°. The orientation of the components fulfil the so called right-hand rule:

Make a fist. Do a "thumbs up" with your right hand (x axis). Point the forefinger (y axis) to the front (make a "pistol" gesture). Now, point the middle finger away from the palm (z axis).

Now let's take a look at the first coordinate system: The Ecliptic Coordinate System. It is widely used for Space Science and for interplanetary missions and can be applied for Solar System centred or planetary tasks. In our last tutorials I was already referring to this system as *ECLIPJ2000* (SPICE naming convention).

Let's take a look at Earth. Our planet revolves around the Sun on an imaginary plane, the so called ecliptic plane. Now, for a coordinate system, we need to define 2 axes; the third one results from the right-hand rule. The x axis is the so called Vernal Equinox: At the beginning of spring, we take the a look at the Sun. The corresponding direction is defined as the x axis (see sketch below). At this very moment, based on the direction to the Sun and the Earth's velocity vector, we get the ecliptic plane. The z axis is perpendicular to this plane (to the Ecliptic north) and the y axis results from the right-hand rule.



Simple top view sketch of the definition of the Ecliptic Coordinate System. The x axis points to the Sun at the beginning of spring. Considering the velocity vector of Earth, the resulting ecliptic plane allows one to define the z axis that is perpendicular to this plane (the z axis points towards the reader). The y axis results from the right-hand rule. The Earth moves anti-clockwise in this sketch. Credit: T. Albin

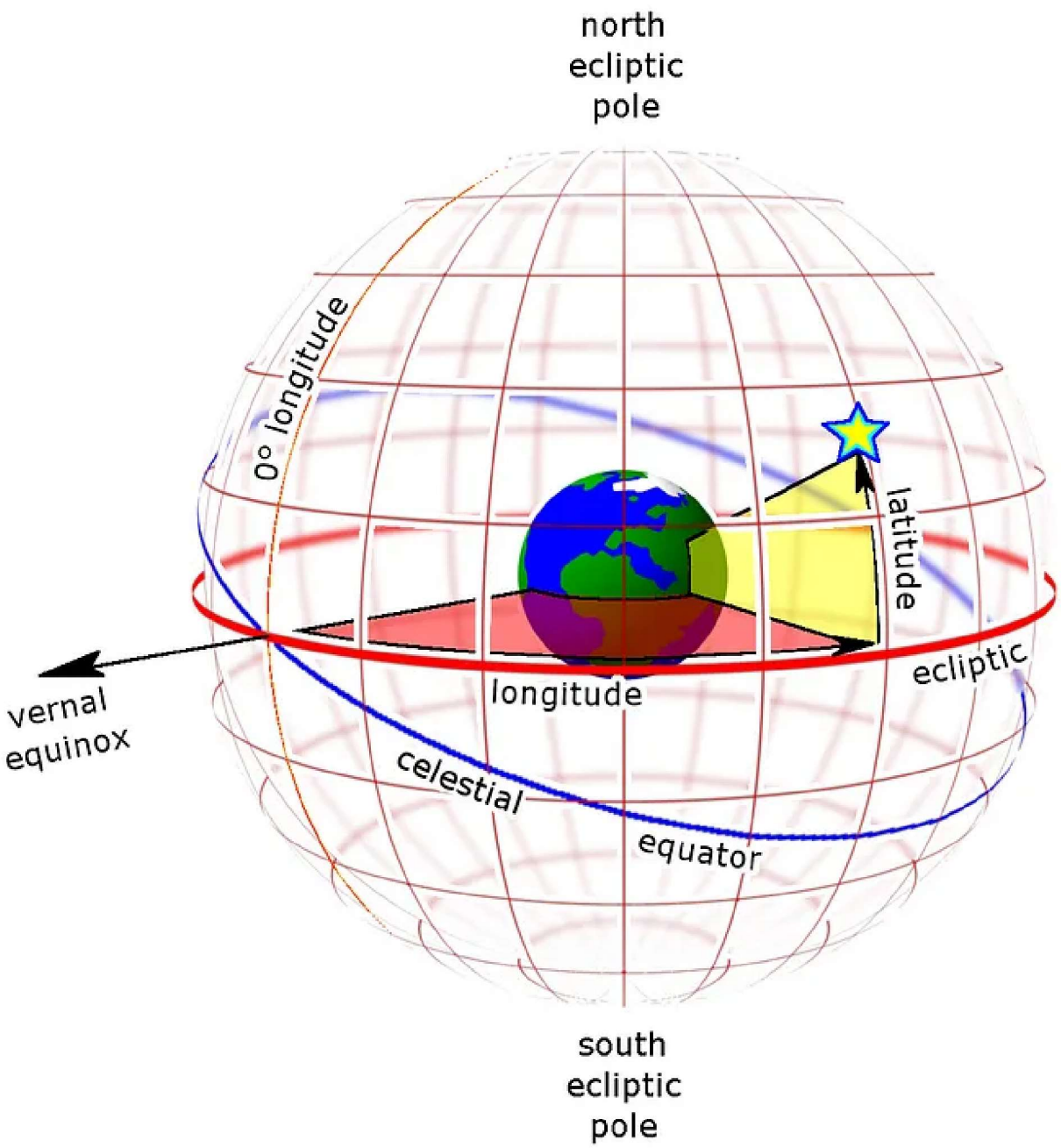
Any 3-dimensional vector can be transformed to spherical coordinates that can be described by 3 parameters: the radius, the azimuth and the inclination. Projecting these values on a unit sphere (sphere with radius = 1) leads to a sky map. The following coordinates describe the position on this map:

- **Longitude** (l): Is defined from 0° to 360°. For the Ecliptic Coordinate System, a longitude of 0° indicates the Vernal Equinox line. All other longitude values are

specified with respect to this line (an imaginary line from one pole to the other.

- **Latitude** (b): Is defined from -90° (south pole of the sphere) to +90° (north pole of the sphere). Values with a latitude of 0° lie on the equator; respectively on the ecliptic plane for the Ecliptic Coordinate System.

This means that l=0° and b=0° (x axis) is the Vernal Equinox point.



Visualisation of the Ecliptic Coordinate System and its projection sphere around the Earth. The red thick circle indicates the ecliptic plane and the blue thick circle represents the equator line of our planet. The angle between the ecliptic plane and the equator is around 23°. Credit: Wikipedia / Wikimedia Commons; License: CC BY-SA 3.0

Now what does J2000 mean? Due to gravitational perturbations of other planets our Earth's orbit "wobbles" a little bit. Consequently, the ecliptic plane changes slightly with time. To avoid a yearly or even daily reference frame update, the ecliptic plane that is being defined at the date J2000 (noon 1st January 2000) is commonly used.

Okay, enough theory. Let's have a look at the programming part. *How can we work with these reference frames? How can we compute positions on the sky? And how does SPICE helps us with that?*

Let's compute and plot a sky map with the coordinates of the Sun, Venus, Mars and our Moon. First, we need to import all necessary modules that were already introduced and used in the previous tutorials: *datetime*, *spiceypy* (SPICE wrapper), *numpy*, *pandas* and *matplotlib* (plotting). Before we can compute any positions, we need the SPICE kernel meta file that contains the relative paths to the relevant kernels. It contains the spk kernel *de432s.bsp* for the planetary positions and *naif0012.tls* for the time conversions and leap seconds. The meta file is loaded with *furnsh*. We set this very moment (*datetime.datetime.now()*) as the date-time object and convert it directly to a corresponding string (line 12). Please consider the following warning from the official *datetime* documentation if you are using *.now()*:

```
Because naive datetime objects are treated by many datetime methods
as local times, it is preferred to use aware datetimes to represent
times in UTC. As such, the recommended way to create an object
representing the current time in UTC is by calling
datetime.now(timezone.utc).
```

Line 15 converts the UTC string to the corresponding Ephemeris time (ET) with the SPICE function *utc2et*.

Part 1/10

For the computation a *pandas* dataframe is defined. This dataframe will contain all resulting coordinates of the ET respectively the UTC time (line 7+8). The positions are computed in a for-loop that iterates through a dictionary of all needed celestial bodies and their NAIF ID code (defined in line 13). Since the loaded spk kernel does not contains the planet's centre of Mars, the barycentre is used (ID: 4).

Part 2/10

Now, the for-loop iterates through the dictionary and computes the directional vector to each celestial object as seen from Earth (line 8–13). The SPICE function *spkezp* is used for this purpose with the following input arguments:

- **targ**: The NAIF ID of the object that shall be observed
- **et**: The ET as computed in the beginning
- **ref**: The reference frame. Here it is the Ecliptic Coordinate System *ECLIPJ2000*
- **abcorr**: The light time correction as described last time (NAIF detailed documentation, if you want to know everything). Here the input *LT+S* indicates that the position determination as seen from the observer (Earth) considers the object's velocity and the travel time of the light.
- **obs**: The NAIF ID of the observer, here: our home planet (399)

Thanks to the SPICE function *recrad*, the direction vectors can be easily converted to spherical coordinates. The longitude and latitude values are the second and third output values, respectively. The first output value is the length (norm) of the vector and is not needed in our case.

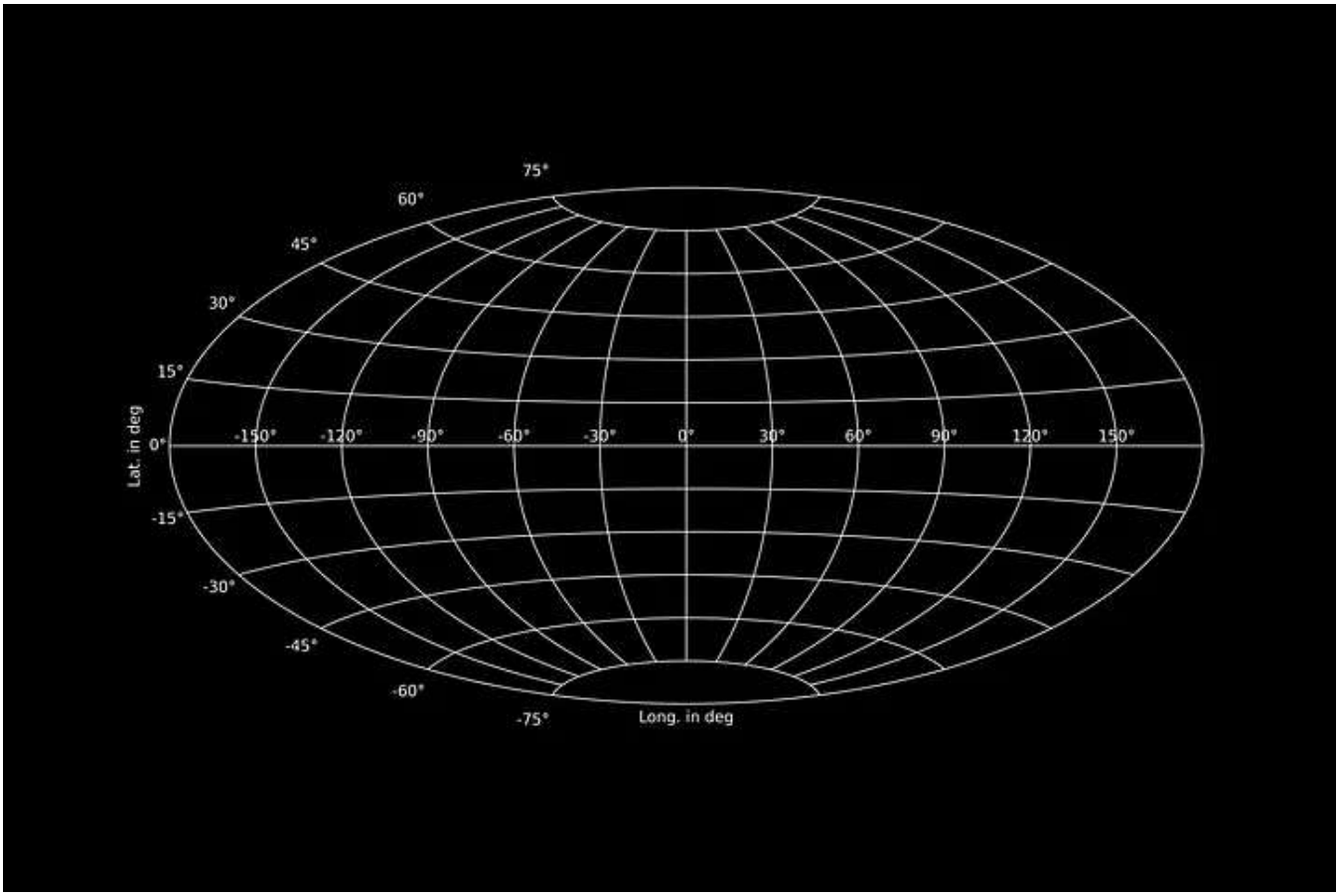
Now we have the coordinates of our objects. *Plotting should be quite easy, right*? We can create a simple *matplotlib* plot and we are done.

Not so fast … A rectangular plot would be quick and dirty and there are better solutions to plot sky coordinates. If you "cut a sphere" and try to project its surface in 2 dimensions one has several projection options, like:

- **Aitoff**
- **Hammer**
- **Mollweide**

Each projection type has its advantages and disadvantages, like representing an equal area map, but not providing correct angular measures (Mollweide). The Aitoff projection provides only a length accuracy and equal area around the meridian and equator, respectively. Hammer compensates these Aitoff issues. An extensive list of projections can be found on the NASA's GISS website. Let's have a look at an empty map with an Aitoff projection. The following Python code creates the figure below:

Part 4/10



Empty Aitoff projection with default matplotlib settings. The latitude ranges from -90° to +90° and the longitude is defined per default from -180° to +180° (counting from left to right). Credit: T. Albin

You can see the latitude values range from -90° to +90° and the longitudes vary between -180° and +180°. As mentioned before however, the longitude values in the Ecliptic Coordinate System range from 0° to 360°. Further, the longitude's values count from right to left. *matplotlib* does not allow one to redefine the projection properties and one cannot easily invert the x axis with a formatting function. Thus, we need to create a small workaround and be a little bit creative. The following coding part shows a for-loop that iterates through all longitude values and creates a new column with longitudes "for the plot":

All angles are given in radians. Values that are larger than pi (180°) exceed the plotting range of *matplotlib.* The modulo operator is applied with pi (*% np.pi*). The remainder is consequently between 0 and pi and should be "on the left side" of the projection map (-180° to 0°), so we need to subtract pi. Finally we multiply the result with -1 to invert the x axis values.

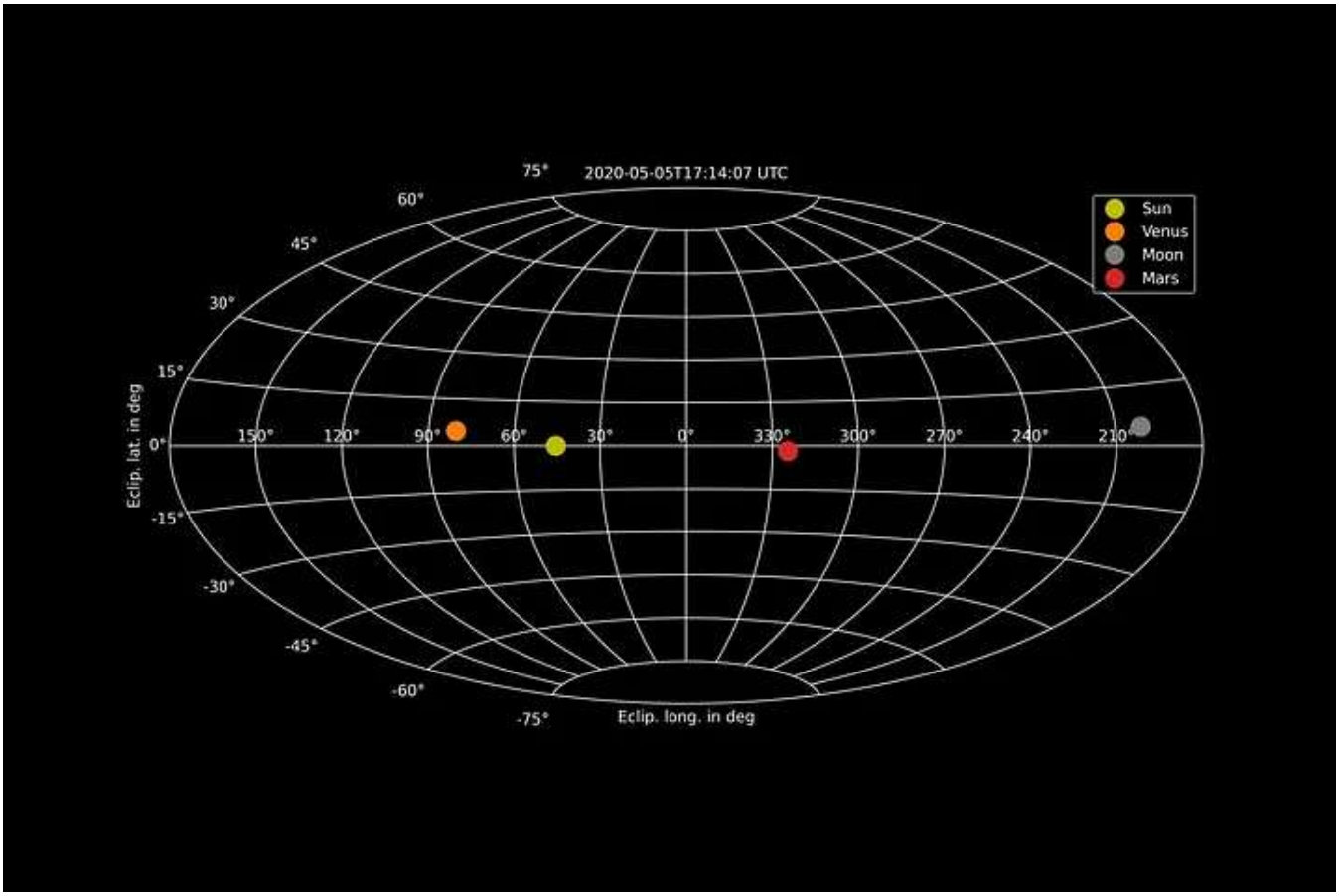
```
-1*((x % np.pi) - np.pi) applies only if x is larger than pi
```

```
-1*x applies else (x is smaller than pi)
```

Part 5/10

Now we can plot the coordinates of the celestial bodies in Ecliptic coordinates. First a dark background is set (line 4) because … space is dark! Line 7 defines the figure and the line after sets the projection. We use the UTC time stamp as a title (line 11) and a simple array defines some colours (line 14) that are used in a for-loop that iterates through all celestial bodies (line 17 to 24). Line 27 to 30 redefine the x ticks and replace the *matplotlib* ticks with the ecliptic longitudes. Finally, the labels are set, a legend and grid are created and the plot is saved. The resulting sky map can be seen below.

Part 6/10



Sky map (Aitoff projection) of the Ecliptic Coordinate System. The Sun, Venus, Mars and Moon are displayed in different colours (legend upper right). The angular width of the objects are not in scale. Credit: T. Albin

The sky map shows the Ecliptic Coordinate System and the positions of the Sun, Venus, Mars and the Moon at around 5 pm on the 5th May 2020. You can see that the planets are located close to the ecliptic plane. The Moon is almost 180° away from the Sun. Have a look outside at the evening: It is almost Full Moon.

Equatorial Coordinate System

Did you consider buying a telescope? It is a calm and inspiring hobby. Either in a group or alone you can explore the planets, galaxies and other cosmic wonders from your backyard (if it is not too light-polluted). But there are a lot of different telescope types and mounts. One common mount type is the so called equatorial mount. This mount is aligned with the rotation axis of the Earth and “rotates with the sky”. Long exposure images, or visual observations without constant re-adjustments are possible with these mounts. Modern mounts have an internal clock combined with an electric motor to compensate the rotation.

The corresponding coordinate system is the so called Equatorial Coordinate System (In SPICE it is simply called *J2000* … a little bit confusing, considering that J2000 is a timestamp), where the x axis points to the Vernal Equinox direction and the z axis is the rotation axis of Earth. Earth's equator is inclined with respect to the Ecliptic plane (around 23°, see 2nd sketch at the beginning) and does not "overlap" with the Ecliptic Coordinate System.

The longitude and latitude in equatorial coordinates are described as:

- **Right ascension**: This is the longitude of the Equatorial Coordinate System and is defined in hours (h), not in degrees, and ranges from 0h to 24h (1h corresponds to 15 degrees).
- **Declination**: This is the latitude and ranges from -90° to +90°. 0° corresponds to the equator line.

Now, let's compute the equatorial coordinates for our celestial bodies of interest. We apply again a for-loop that iterates through all bodies and we apply the same functions as before to compute the directional vector and the coordinates in the Equatorial Coordinate System.

Part 7/10

To visualise the resulting distortion between the Ecliptic and Equatorial Coordinate System, we add the ecliptic plane in our computations. For this purpose, an additional *pandas* dataframe is created (line 6). Line 11 adds the longitude coordinates in *ECLIPJ2000*, from 0 to 2*pi (360°), as an array. Line 12 adds the latitude values. Actually, the coordinates should be 0°, in this case we need the spherical coordinate convention. Line 18 to 23 converts the spherical ecliptic coordinates to directional vectors using the SPICE function *sphrec*. The function needs the distance *r* (here: unit sphere with radius 1), the latitude (*colat*) and longitude (*lon*) values. The returned vectors are the directional vectors of the ecliptic plane in a vectorised Ecliptic Coordinate System (x, y, z components).

Part 8/10

In a next step we need to transform these vectors to the vectorised Equatorial Coordinate System (x, y, z components). We use the SPICE function *pxform* that computes a 3x3 transformation matrix between both coordinate systems. The function requires the following input:

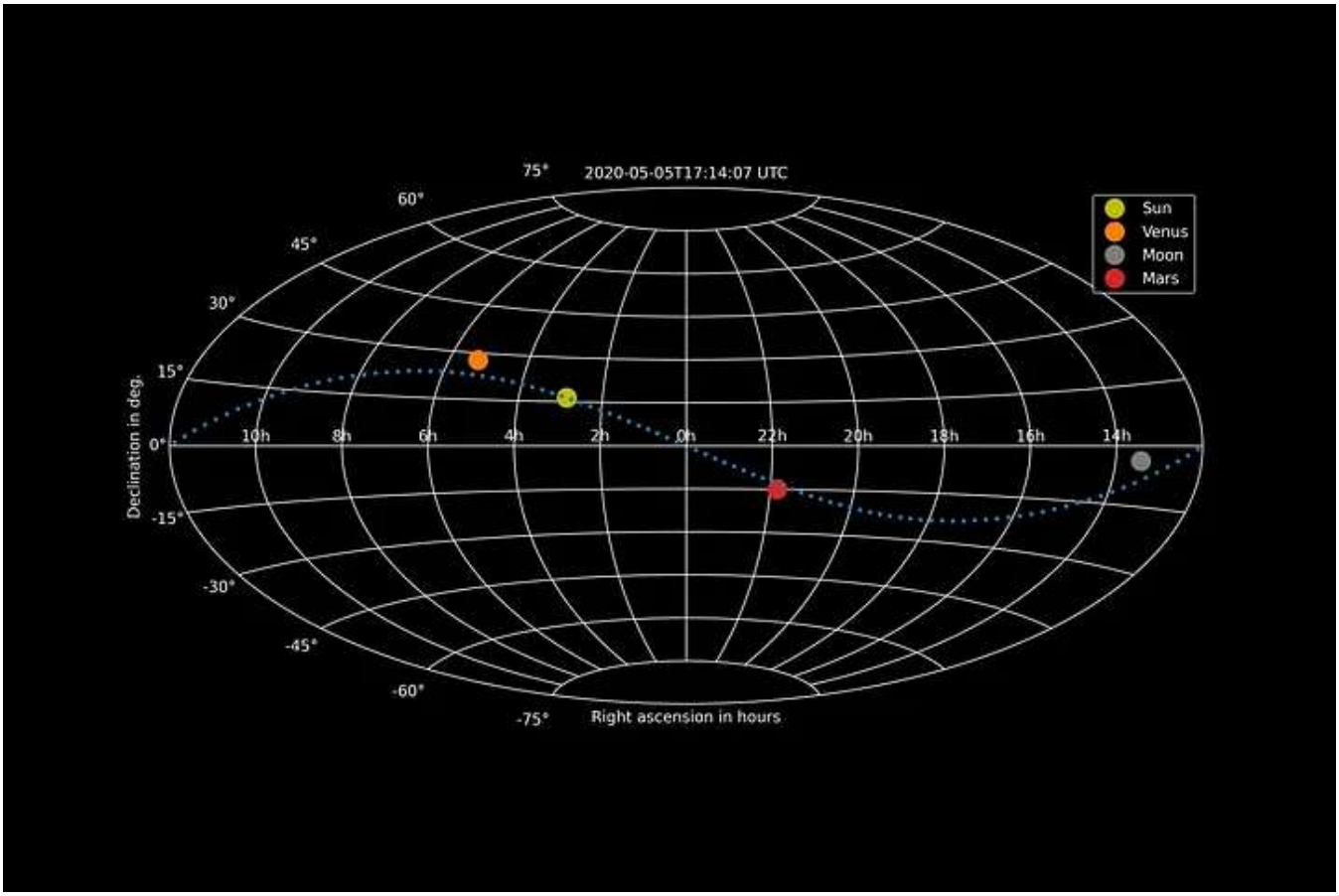
- **fromstr**: (Note: the suffix **str** applies only for the *spiceypy* library) The coordinate system's name to transform from (here: ECLIPJ2000)
- **tostr**: (Note: the suffix **str** applies only for the *spiceypy* library) The coordinate system's name to transform to (here: *J2000*)
- **et**: The ET between both transformations. For inertial coordinate systems this value can be any ET (systems that do not change their orientation / definition in time. In future tutorials we will encounter rotational coordinate systems, where this input parameter must be considered)

With the transformation matrix we can transform the vectors from *ECLIPJ2000* to *J2000* as shown in line 10 and 11. The matrix is applied with a dot product on the vectors. Afterwards (line 15–24) the vectors in the vectorised Equatorial Coordinate System can be transformed to right ascension and declination values using the SPICE function *recrad*.

Part 9/10

Now we can plot the celestial objects and add the ecliptic plane in equatorial coordinates (line 17–19) as a blue dotted line. Similarly as before, we set some formatting properties and change the x ticks to right ascension hours (line 22–25). The resulting figure is shown below.

Part 10/10



Sky map (Aitoff projection) of the Equatorial Coordinate System. The Sun, Venus, Mars and Moon are displayed in different colors (legend upper right). The angular width of the objects are not in scale. The blue dotted line represents the ecliptic plane in this coordinate system. Credit: T. Albin

You can see that the ecliptic plane describes a sinusoidal curve on the Equatorial Coordinate System. The Sun and the planets are again close to the ecliptic plane. The Sun is at around 3 hours and has a declination of more than 15 degrees. During the year, the Sun "moves" along the blue dotted line from right to left. You can see seasonal variations: The Sun crossed already the 0h / 0° Vernal Equinox point and moves "higher" in declination. This mean that the readers of you who live on the northern hemisphere will have longer days until the beginning of Summer. Afterwards, the Sun moves downwards and the days get shorter.

Conclusion

Today we learned how to compute the coordinates of celestial bodies for different coordinate systems. This knowledge will be required for future tutorials, where we will make observation planning and asteroid tracking.

Astronomers and radio telescope operators need to compute these coordinates precisely, e.g., to find objects, for the determination of an asteroids orbits or to track and communicate with satellites and spacecraft missions. If you want to get a "better feeling" about this topic with more flexibility and visualisations (but with less Python computations) have a look at the Open Source planetarium software *Stellarium*.

Thomas

Pyhton Space Science Programming Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play